付録 — 逆引き: やりたいことからコマンドを探す

「サービスを止めたい」「古いバックアップを消したい」から、コマンドと設定を 214 個引く。本文に出てくる所と、合わせて覚えるコマンドつき

付録: 逆引き — やりたいことからコマンドを探す

この資料について

09 付録「出てくるコマンドと設定の一覧」は、「この `-StartupType` は何だっけ？」のように、**コマンドから意味を引く**ための物です。この付録は、その反対です。「**サービスを止めたい**」「**古いバックアップを消したい**」「**ポートにつながるか確かめたい**」のような、**やりたいことから、コマンドや設定の書き方を引く**ための物です。

- **全部で 214 個**の「やりたいこと」を、**11 の分類**に分けています。下の「分類の一覧」から選ぶか、HTML 版（`onprem\html`）で `Ctrl` ＋ `F` を押して、やりたいことの言葉（「止める」「消す」「確かめる」など）で探してください。
- **「書き方」の列:** そのまま使える形の、短いコマンドです。名前（`webapp-prod`・`D:\webapp\…`）は、04 の 4.1 の決まりに合わせています。自分の環境に合わせて書きかえます。
- **「ひとこと」の列:** 気をつけることや、結果の見方です。くわしい意味は、09 付録の一覧と、本文の「▶ ここで初めて出てくるコード」の箱にあります。
- **「出てくる所」の列:** そのコマンドが、00〜08 と「参考」で初めて出てくる資料と章です（187 個）。
- **「＋ 合わせて覚える」:** 本文のコマンドと一緒に覚えておくと、困った時に役立つコマンドです（27 個）。この付録で足しました。本番のサーバーで打つ前に、`Get-Help コマンド名 -Examples` などで、くわしい使い方を確かめてください。

> **ヒント:** 本番のサーバーで、止める・消す・戻すコマンドを打つ前には、まず「見る」コマンド（`Get-Service`・`Get-ChildItem`・`\l` など）で、対象が合っているかを確かめてください。

分類の一覧

分類	数	分類	数
PowerShell の基本の操作	56	決まった時刻に動かす（タスク スケジューラ）	10

分類	数	分類	数
サーバーの様子を見る（OS・CPU・メモリ・ディスク）	21	PostgreSQL と psql	34
サービス（ずっと動くプログラム）	13	バックアップと戻し方	9
IIS（Web サーバー）	18	アプリを持ち込んで動かす（Rust 版・Django 版）	15
ファイアウォールとネットワーク	20	正規表現と日時の書式	9
フォルダの権限とコピー	9		

PowerShell の基本の操作

サーバーの上で、管理者の PowerShell から打つ時の、基本の書き方です。

やりたいこと	書き方	ひとこと	出てくる所
管理者として動いているか確かめる	`([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)`	True なら管理者	01 3. PowerShell の基本
コマンドの使い方を見る	`Get-Help Get-Service -Examples`	例が出る	01 3. PowerShell の基本
コマンドがあるか・どこにあるか	`Get-Command psql`		01 3. PowerShell の基本
前に打ったコマンドの一覧	`Get-History`		01 3. PowerShell の基本
今いるフォルダを移る	`cd D:\webapp\prod`		05 7. ビルドして、まとめる

やりたいこと	書き方	ひとこと	出てくる所
いったん移って、あとで戻る	`Push-Location D:\webapp` `Pop-Location`		05 9. サーバーに置き、Windows サービスにする
フォルダの中を見る	`Get-ChildItem D:\webapp\prod\logs`		01 3. PowerShell の基本
大きい順に見る	`Get-ChildItem -Recurse -File | Sort-Object Length -Descending | Select-Object -First 20 FullName, Length`		01 3. PowerShell の基本
フォルダを作る	`New-Item -ItemType Directory -Force D:\webapp\prod\logs`	もうあってもよい	04 4. フォルダとアカウントの準備
ファイルの中を見る	`Get-Content D:\webapp\prod\logs\app.log -Tail 50`	最後の 50 行	01 3. PowerShell の基本
増える行を見続ける	`Get-Content app.log -Tail 50 -Wait`	Ctrl＋C で止める	01 3. PowerShell の基本
文字を探す	`Select-String -Path *.log -Pattern "ERROR"`		01 3. PowerShell の基本
ファイルに書く	`Set-Content -Path a.txt -Value "…" -Encoding utf8`		08 この資料について

コピー・名前の変更・削除	`Copy-Item a b`
	`Rename-Item a b`
	`Remove-Item a`
			04 6. アプリを作る

あるか調べる	`Test-Path D:\webapp\prod\app\webapp.exe`		05 13. 動作確認・更新・切り戻し
場所をつなぐ	`Join-Path $PSScriptRoot "dev.env"`		04 11. 毎晩のバックアップ
ファイルやフォルダの情報	`Get-Item D:\webapp\prod\app\webapp.exe`		08 この資料について

やりたいこと	書き方	ひとこと	出てくる所
変数	`$dir = "D:\webapp\backup"`		01 3. PowerShell の基本
環境変数	`$env:Path` `$env:Path += ";C:\Program Files\PostgreSQL\18\bin"`	この窓だけ	01 3. PowerShell の基本
文字の中に変数	`"サービス: $name"` `"ポート $($t.Port)"`		01 3. PowerShell の基本
そのままの文字	`'C:\Program Files'`		01 3. PowerShell の基本
空白の入った場所のプログラム	`& "C:\Program Files\PostgreSQL\18\bin\psql.exe" --version`		00 0. はじめの一歩: コンピューターの中身
今いるフォルダのスクリプト	`.\backup.ps1`		01 3. PowerShell の基本
結果を次に渡す	`Get-Service | Where-Object Status -eq Running`		00 3. OS
条件でしぼる	`Where-Object { $_.LastWriteTime -lt $limit }`	$_ は流れてきた 1 つ	01 3. PowerShell の基本
1 つずつ処理	`ForEach-Object { $_.FullName }`		06 5. 症状別の詳しい説明
列を選ぶ・数をしぼる	`Select-Object Name, Status -First 10`		00 3. OS
計算した列を作る	`Select-Object Name, @{ Name = 'MB'; Expression = { [int]($_.Length / 1MB) } }`		01 3. PowerShell の基本
並べかえる	`Sort-Object LastWriteTime`		00 4. ネットワークの基礎
数える・合計	`Measure-Object Length -Sum`	.Sum	06 5. 症状別の詳しい説明

やりたいこと	書き方	ひとこと	出てくる所
表で見る・全部の項目を見る	`Format-Table -AutoSize` `Format-List *`		04 3. サーバーに入る・最初の確認
出力を捨てる	`… | Out-Null`		04 4. フォルダとアカウントの準備
文字を作る（書式）	`"全体 {0:N1} GB" -f ($x / 1GB)`		01 8. 重い・遅いを調べる
日時を文字にする	`Get-Date -Format "yyyyMMdd_HHmm"`		04 11. 毎晩のバックアップ
何日前の日時	`(Get-Date).AddDays(-14)`		01 7. イベントビューアー
文字から日時を作る	`[datetime]::ParseExact("202611050200", "yyyyMMdd HHmm", $null)`		08 この資料について
くり返す	`foreach ($svc in @("W3SVC", "webapp-prod")) { … }`		04 4. フォルダとアカウントの準備
条件で分ける	`if ($LASTEXITCODE -ne 0) { … } else { … }`		04 11. 毎晩のバックアップ
比べる	`-eq` `-ne` `-lt` `-gt` `-like "webapp*"` `-match '^\d+$'`		01 3. PowerShell の基本
エラーで止める	`$ErrorActionPreference = "Stop"`		04 11. 毎晩のバックアップ
エラーを受け止める	`try { … } catch { $_.Exception.Message } finally { … }`		04 11. 毎晩のバックアップ
エラーを起こす	`throw "pg_dump が失敗しました"`		04 11. 毎晩のバックアップ
スクリプトを終わる	`exit 1`		04 11. 毎晩のバックアップ

やりたいこと	書き方	ひとこと	出てくる所
関数を作る	`function Show-Result($Name, $Ok) { … }`		06 2. 切り分け: 手前から順に確かめる
引数を受け取る	`param([ValidateSet("prod", "stg")] [string] $Env = "prod")`		05 9. サーバーに置き、Windows サービスにする
入力してもらう	`Read-Host "続けますか (y/n)"`		参考 4. 証明書をサーバーに入れる
記録を全部ファイルに残す	`Start-Transcript D:\work\log.txt` `Stop-Transcript`		04 11. 毎晩のバックアップ
長いコマンドを続ける（行の最後）	`` ` ``		01 10. Windows Defender ファイアウォール
XML を読む	`[xml](Get-Content x.xml -Encoding UTF8)`		05 9. サーバーに置き、Windows サービスにする
外から取ってきたファイルを使えるようにする	`Unblock-File .\winsw.exe`		01 3. PowerShell の基本
スクリプトを動かせる設定か見る	`Get-ExecutionPolicy -List`		01 3. PowerShell の基本
本当には動かさずに試す	`Remove-Item D:\webapp\backup\*.dump -WhatIf`	何が消えるかだけ表示	＋ 合わせて覚える
確かめを聞いてもらう	`Remove-Item … -Confirm`	1 つずつ聞かれる	＋ 合わせて覚える
名前でプログラムを止める	`Stop-Process -Name webapp -Force`	サービスは Stop-Service で	＋ 合わせて覚える
管理者として PowerShell を開く	`Start-Process powershell -Verb RunAs`		＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
ファイルの権限を見る	`Get-Acl D:\webapp\prod\logs | Format-List`	icacls と同じ物を別の形で	＋ 合わせて覚える

サーバーの様子を見る（OS・CPU・メモリ・ディスク）

サーバーが元気かを調べるコマンドです。

やりたいこと	書き方	ひとこと	出てくる所
OS の版など	`Get-ComputerInfo -Property OsName, OsVersion`		04 3. サーバーに入る・最初の確認
コンピューターの名前	`hostname`		00 0. はじめの一歩: コンピューターの中身
今の利用者	`whoami`		00 0. はじめの一歩: コンピューターの中身
ログインしている人	`quser`		01 1. サーバーに入る
最後に起動した日時	`(Get-CimInstance Win32_OperatingSystem).LastBootUpTime`		01 11. Windows Update と再起動
入っている更新プログラム	`Get-HotFix | Sort-Object InstalledOn -Descending`		01 11. Windows Update と再起動
CPU を多く使うプログラム	`Get-Process | Sort-Object CPU -Descending | Select-Object -First 10`		01 8. 重い・遅いを調べる
メモリの全体と空き	`Get-CimInstance Win32_OperatingSystem`	.TotalVisibleMemorySize・.FreePhysicalMemory	01 6. サービス

やりたいこと	書き方	ひとこと	出てくる所
ディスクの空き	`Get-Volume | Select-Object DriveLetter, SizeRemaining, Size`		01 8. 重い・遅いを調べる
ドライブの一覧	`Get-PSDrive -PSProvider FileSystem`		00 0. はじめの一歩: コンピューターの中身
時間帯を見る・変える	`Get-TimeZone` `Set-TimeZone -Id "Tokyo Standard Time"`		04 3. サーバーに入る・最初の確認
時計の合わせ先を見る	`w32tm /query /status`		04 3. サーバーに入る・最初の確認
Windows の機能を見る・入れる	`Get-WindowsFeature Web-*` `Install-WindowsFeature Web-Server -IncludeManagementTools`		04 8. 受付
イベントログ（エラー）を見る	`Get-WinEvent -LogName System -MaxEvents 50`		01 7. イベントビューアー
エラーだけを見る	`Get-WinEvent -FilterHashtable @{ LogName = 'Application'; Level = 2 } -MaxEvents 20`	Level 2 はエラー	01 7. イベントビューアー
ローカルの利用者・グループ	`Get-LocalUser` `Get-LocalGroupMember Administrators`		01 4. アカウントとグループ
人を切断する	`logoff 2`	quser の ID	01 1. サーバーに入る
CPU・メモリの使い方を数秒おきに見る	`Get-Counter '\Processor(_Total)\% Processor Time' -SampleInterval 2 -MaxSamples 5`		＋ 合わせて覚える

やりたいこと	書き方	ひとこと	出てくる所
サーバーを再起動する	`Restart-Computer`	利用者に知らせてから	＋ 合わせて覚える
イベントログの種類の一覧	`Get-WinEvent -ListLog * | Where-Object RecordCount -gt 0`		＋ 合わせて覚える
プログラムの一覧（古い方法）	`tasklist`		＋ 合わせて覚える

サービス（ずっと動くプログラム）

アプリ・IIS・PostgreSQL は、Windows のサービスとして動かします。

やりたいこと	書き方	ひとこと	出てくる所
サービスの一覧	`Get-Service`		00 3. OS
名前でしぼる	`Get-Service -Name *postgres*`		00 3. OS

動かす・止める・動かし直す	`Start-Service webapp-prod`
	`Stop-Service webapp-prod`
	`Restart-Service webapp-prod`
			01 6. サービス

起動の種類を変える	`Set-Service webapp-prod -StartupType Automatic`		01 6. サービス
自動なのに止まっている物	`Get-CimInstance Win32_Service | Where-Object { $.StartMode -eq "Auto" -and $.State -ne "Running" }`		06 5. 症状別の詳しい説明
サービスの設定を見る	`sc.exe qc webapp-prod`		01 6. サービス

WinSW でサービスにする	`.\webapp-prod-service.exe install`
	`start`
	`stop`
	`uninstall`
		同じフォルダの XML を読む	04 7. アプリを Windows サービスにする

動かすプログラム（WinSW）	`<executable>D:\webapp\prod\app\webapp.exe</executable>`		04 7. アプリを Windows サービスにする
渡す環境変数（WinSW）	`<env name="PORT" value="8080" />`		04 7. アプリを Windows サービスにする

やりたいこと	書き方	ひとこと	出てくる所
先に動かすサービス（WinSW）	`<depend>postgresql-x64-18</depend>`		04 7. アプリを Windows サービスにする
落ちた時に動かし直す（WinSW）	`<onfailure action="restart" delay="10 sec" />`		04 7. アプリを Windows サービスにする
ログの残し方（WinSW）	`<log mode="roll-by-size">`		04 7. アプリを Windows サービスにする
自動で起動（WinSW）	`<startmode>Automatic</startmode>`		04 7. アプリを Windows サービスにする

IIS（Web サーバー）

社員のブラウザーからの要求を受けて、アプリへ取り次ぐ IIS の操作です。

やりたいこと	書き方	ひとこと	出てくる所
IIS のコマンドを使えるようにする	`Import-Module IISAdministration`		04 8. 受付
サイトの一覧	`Get-IISSite`		04 8. 受付
サイトを作る	`New-IISSite -Name webapp-prod -PhysicalPath D:\webapp\prod\iis -BindingInformation "*:80:webapp.example.local"`		04 8. 受付
サイトを止める	`Stop-IISSite -Name "Default Web Site" -Confirm:$false`		04 8. 受付
待ち受けの組み合わせを見る	`Get-IISSiteBinding -Name webapp-prod`		参考 5. IIS に 443 番のバインドを足す
待ち受けを足す	`New-IISSiteBinding -Name webapp-prod -BindingInformation "*:443:…" -Protocol https`		参考 5. IIS に 443 番のバインドを足す

やりたいこと	書き方	ひとこと	出てくる所
アプリへ取り次ぐ決まり	`<rule name="to-webapp-prod" stopProcessing="true">`		04 8. 受付
どの URL に合わせるか	`<match url="(.*)" />`		04 8. 受付
取り次ぐ先	`<action type="Rewrite" url="http://127.0.0.1:8080/{R:1}" />`		04 8. 受付
合った部分を使う	`{R:1}`		04 8. 受付
送れる大きさの上限	`<requestLimits maxAllowedContentLength="104857600" />`	100MB	04 8. 受付
HTTPS へ移す	`<action type="Redirect" url="https://{HTTP_HOST}/{R:1}" redirectType="Found" />`		参考 9. 80 番で来たお願いを、443 番へ案内する
条件を付ける	`<conditions><add input="{HTTPS}" pattern="off" /></conditions>`		参考 9. 80 番で来たお願いを、443 番へ案内する
見出しを付けて取り次ぐ	`<serverVariables><set name="HTTP_X_FORWARDED_PROTO" value="https" /></serverVariables>`		参考 9. 80 番で来たお願いを、443 番へ案内する
IIS のログを見る	`Get-ChildItem C:\inetpub\logs\LogFiles\W3SVC -Filter .log | Sort-Object LastWriteTime | Select-Object -Last 1`		04 12. 定期処理・ログ・監視
500 番台の行を探す	`Select-String -Path $log.FullName -Pattern " 5\d\d \d+ \d+ \d+$"`		06 3. 記録
IIS を動かし直す	`iisreset`		＋ 合わせて覚える
サイトを動かす	`Start-IISSite -Name webapp-prod`		＋ 合わせて覚える

ファイアウォールとネットワーク

どのポートを開けるか、つながるかを確かめるコマンドです。

やりたいこと	書き方	ひとこと	出てくる所
決まりを作る	`New-NetFirewallRule -DisplayName "webapp HTTP 80" -Direction Inbound -Protocol TCP -LocalPort 80 -Action Allow`		01 10. Windows Defender ファイアウォール
決まりの一覧	`Get-NetFirewallRule -Enabled True -Direction Inbound`		01 10. Windows Defender ファイアウォール
ポートで決まりを探す	`Get-NetFirewallPortFilter | Where-Object { $_.LocalPort -in 80, 443 } | Get-NetFirewallRule`		04 9. ファイアウォール
決まりを止める	`Disable-NetFirewallRule -DisplayName "…"`		04 9. ファイアウォール
ファイアウォールの状態	`Get-NetFirewallProfile`		01 10. Windows Defender ファイアウォール
ネットワークの種類（ドメイン・プライベート）	`Get-NetConnectionProfile`		01 10. Windows Defender ファイアウォール
IP アドレスなど	`Get-NetIPConfiguration`		00 4. ネットワークの基礎
名前から IP アドレスを調べる	`Resolve-DnsName webapp.example.local`		00 4. ネットワークの基礎
ポートにつながるか	`Test-NetConnection 127.0.0.1 -Port 5432`	TcpTestSucceeded	00 4. ネットワークの基礎
待ち受けているポート	`Get-NetTCPConnection -State Listen -LocalPort 8080`	.OwningProcess が番号	00 4. ネットワークの基礎

やりたいこと	書き方	ひとこと	出てくる所
ページを開いて状態コードを見る	`(Invoke-WebRequest http://webapp.example.local/health -UseBasicParsing).StatusCode`		00 5. Web アプリの仕組み
名前を付けて開く	`Invoke-WebRequest http://127.0.0.1:8080/health -Headers @{ Host = "webapp.example.local" }`		00 5. Web アプリの仕組み
要求と答えの形	`GET /requests HTTP/1.1` `HTTP/1.1 200 OK`		00 5. Web アプリの仕組み
ポートを使っているプログラム（古い方法）	`netstat -ano | findstr :8080`		＋ 合わせて覚える
相手に届くか（ping）	`Test-Connection webapp.example.local -Count 2`		＋ 合わせて覚える
通り道を調べる	`tracert webapp.example.local`		＋ 合わせて覚える
IP アドレスの全部を見る（古い方法）	`ipconfig /all`		＋ 合わせて覚える
名前の覚え書きを消す	`Clear-DnsClientCache`	DNS を直した後に	＋ 合わせて覚える
ネットワークの口の一覧	`Get-NetAdapter`		＋ 合わせて覚える
共有フォルダの一覧	`Get-SmbShare`		＋ 合わせて覚える

フォルダの権限とコピー

アプリのフォルダを、サービスの利用者だけが書けるようにします。

やりたいこと	書き方	ひとこと	出てくる所
権限を見る	`icacls D:\webapp\prod\logs`		01 5. ファイルとアクセス許可
権限を与える	`icacls D:\webapp\prod\logs /grant "NT SERVICE\webapp-prod:(OI)(CI)M"`	M は変更	01 5. ファイルとアクセス許可

やりたいこと	書き方	ひとこと	出てくる所
引き継ぎを外す	`icacls D:\webapp\prod\app /inheritance:r`		01 5. ファイルとアクセス許可
権限を外す	`icacls D:\webapp\prod /remove "Users"`		01 5. ファイルとアクセス許可
フォルダごとコピー	`robocopy backend out\app /E /XD .venv __pycache__ /XF *.pyc`		05 7. ビルドして、まとめる
失敗した時のやり直し	`robocopy 元 先 /R:2 /W:5`		01 12. ファイルを持ち込む
ZIP にまとめる・ほどく	`Compress-Archive -Path "$out\*" -DestinationPath app.zip -Force` `Expand-Archive app.zip D:\webapp\prod\app`		04 6. アプリを作る
指紋で同じファイルか確かめる	`Get-FileHash app.zip -Algorithm SHA256`		01 12. ファイルを持ち込む
何でも合う（ワイルドカード）	`*.log` `webapp_prod_*.dump`		04 6. アプリを作る

決まった時刻に動かす（タスク スケジューラ）

バックアップなどを、毎日決まった時刻に動かす仕組みです。

やりたいこと	書き方	ひとこと	出てくる所
動かす物を決める	`New-ScheduledTaskAction -Execute "powershell.exe" -Argument "-NoProfile -File D:\webapp\tools\backup.ps1"`		04 11. 毎晩のバックアップ
いつ動かすか決める	`New-ScheduledTaskTrigger -Daily -At 2:00`		04 11. 毎晩のバックアップ
だれとして動かすか	`New-ScheduledTaskPrincipal -UserId "SYSTEM" -RunLevel Highest`		04 11. 毎晩のバックアップ
細かい設定	`New-ScheduledTaskSettingsSet -ExecutionTimeLimit (New-TimeSpan -Hours 1)`		04 11. 毎晩のバックアップ

やりたいこと	書き方	ひとこと	出てくる所
登録する	`Register-ScheduledTask -TaskName "webapp-backup" -Action $action -Trigger $trigger -Principal $p`		04 11. 毎晩のバックアップ
今すぐ動かす	`Start-ScheduledTask -TaskName "webapp-backup"`		01 9. タスク スケジューラ
最後の結果を見る	`Get-ScheduledTaskInfo -TaskName "webapp-backup"`	LastTaskResult 0 なら成功	01 9. タスク スケジューラ
登録の一覧	`Get-ScheduledTask -TaskPath \`		01 9. タスク スケジューラ
一時的に止める・戻す	`Disable-ScheduledTask -TaskName webapp-backup` `Enable-ScheduledTask -TaskName webapp-backup`		＋ 合わせて覚える
登録を消す	`Unregister-ScheduledTask -TaskName webapp-backup -Confirm:$false`		＋ 合わせて覚える

PostgreSQL と psql

データベースにつないで、SQL を打つ道具 psql の使い方と、よく使う SQL です。

やりたいこと	書き方	ひとこと	出てくる所
つなぐ	`psql -h 127.0.0.1 -p 5432 -U postgres -d postgres`		02 2. ログインする
SQL を 1 回だけ動かす	`psql -h 127.0.0.1 -U webapp_stg_app -d webapp_stg -c "SELECT now();"`		02 2. ログインする
SQL のファイルを流す	`psql -h 127.0.0.1 -U postgres -d practice -f setup.sql`		02 2. ログインする
版を見る	`psql --version`		04 5. データベース
データベースの一覧	`\l`		04 5. データベース
つなぐ先を変える	`\c practice`		02 3. 練習用のデータベースを作る

やりたいこと	書き方	ひとこと	出てくる所
表の一覧・表の形	`\dt` `\d requests`		＋ 合わせて覚える
利用者（ロール）の一覧	`\du`		04 5. データベース
パスワードを変える	`\password webapp_prod_app`	画面に出ない	02 9. ユーザー
縦に並べて見る	`\x`		＋ 合わせて覚える
終わる	`\q`		04 5. データベース
データベースを作る	`CREATE DATABASE webapp_prod OWNER webapp_prod_app ENCODING 'UTF8'`		02 3. 練習用のデータベースを作る
利用者を作る	`CREATE ROLE webapp_prod_app LOGIN PASSWORD '…'`		02 9. ユーザー
権限を与える・取り上げる	`GRANT SELECT ON ALL TABLES IN SCHEMA public TO webapp_readonly` `REVOKE …`		02 9. ユーザー
読む	`SELECT id, title, status FROM requests WHERE status = '回覧中' ORDER BY id DESC LIMIT 20`		00 7. データベースの基礎
全部の列	`SELECT * FROM departments`		02 4. データを見る
文字の一部で探す	`WHERE name LIKE '%田%'`		02 4. データを見る
数える	`SELECT status, count(*) FROM requests GROUP BY status`		02 4. データを見る
表をつなぐ	`SELECT r.title, e.name FROM requests r JOIN employees e ON e.id = r.requester_id`		02 5. テーブルをつなぐ

入れる・変える・消す	`INSERT INTO … VALUES (…)`
	`UPDATE … SET … WHERE …`
	`DELETE FROM … WHERE …`
		WHERE を忘れない	02 3. 練習用のデータベースを作る

やりたいこと	書き方	ひとこと	出てくる所
まとめて成功・失敗にする	`BEGIN; … COMMIT;` `ROLLBACK;`		02 7. トランザクション
今の利用者とデータベース	`SELECT current_user, current_database();`		02 2. ログインする
データベースの大きさ	`SELECT pg_size_pretty(pg_database_size('webapp_prod'));`		02 12. 運用で使う SQL
表の大きさ	`pg_total_relation_size('requests')`		02 12. 運用で使う SQL
つながりの一覧	`SELECT pid, usename, state FROM pg_stat_activity;`		02 12. 運用で使う SQL
つながりを切る	`SELECT pg_terminate_backend(12345);`		02 12. 運用で使う SQL
つないでよい相手の設定を見る	`Get-Content D:\pgsql\18\data\pg_hba.conf | Select-String -Pattern "^\s*host"`		04 5. データベース
設定を読み直す	`SELECT pg_reload_conf();`		00 7. データベースの基礎
SQL のコメント	`-- メモ`		00 7. データベースの基礎
SQL にかかった時間を表示する	`\timing`	もう一度打つと消える	＋ 合わせて覚える
どう探しているか（時間つき）を見る	`EXPLAIN ANALYZE SELECT …`	本当に動かすので UPDATE には使わない	02 4. データを見る
表を整理して統計を新しくする	`VACUUM ANALYZE requests`		＋ 合わせて覚える
設定ファイルの場所を見る	`SHOW config_file;` `SHOW hba_file;`		02 10. 設定ファイル
全部のデータベースと利用者を書き出す	`pg_dumpall -h 127.0.0.1 -U postgres -f all.sql`		＋ 合わせて覚える

バックアップと戻し方

データベースとファイルのバックアップを取り、戻せるかを確かめます。

やりたいこと	書き方	ひとこと	出てくる所
データベースを書き出す	`pg_dump -h 127.0.0.1 -U postgres -Fc -f D:\webapp\backup\db\webapp_prod.dump webapp_prod`	-Fc は PostgreSQL 用の形	02 11. バックアップとリストア
バックアップの中身を見る	`pg_restore --list webapp_prod.dump`		02 11. バックアップとリストア
別のデータベースに戻して確かめる	`pg_restore -h 127.0.0.1 -U postgres -d webapp_restore_test --no-owner --no-privileges webapp_prod.dump`		04 11. 毎晩のバックアップ
同じデータベースに戻す	`pg_restore -d webapp_prod --clean --if-exists webapp_prod.dump`	今のデータは消える	04 13. 動作確認・本番の開始・更新・切り戻し
失敗したら止める	`if ($LASTEXITCODE -ne 0) { throw "pg_dump が失敗しました" }`		04 11. 毎晩のバックアップ
日付入りの名前にする	`"webapp_prod_$(Get-Date -Format yyyyMMdd_HHmm).dump"`		04 13. 動作確認・本番の開始・更新・切り戻し
古いバックアップを消す	`Get-ChildItem $dir -Filter "webapp_prod_*.dump" | Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-14) } | Remove-Item`		04 11. 毎晩のバックアップ
いちばん新しい物を選ぶ	`Get-ChildItem *.dump | Sort-Object LastWriteTime | Select-Object -Last 1`		01 3. PowerShell の基本
ファイルの日時を変える（試し）	`(Get-Item $file).LastWriteTime = (Get-Date).AddDays(-20)`		08 この資料について

アプリを持ち込んで動かす（Rust 版・Django 版）

開発用のパソコンで作った物を、サーバーに置いて動かすまでのコマンドです。

やりたいこと	書き方	ひとこと	出てくる所
Rust 版を本番用に作る	`cargo build --release --locked`		04 6. アプリを作る
CSS などの束を作る	`topcoat asset bundle --release`		04 6. アプリを作る
C の部品を中に入れて作る	`rustflags = ["-C", "target-feature=+crt-static"]`	.cargo/config.toml	04 6. アプリを作る
画面側を作る（Django 版）	`npm ci` `npm run build`		05 7. ビルドして、まとめる
Python の仮想環境を作る	`py -m venv D:\webapp\stg\venv`		05 4. Python を入れる
部品を入れる	`D:\webapp\stg\venv\Scripts\python.exe -m pip install -r requirements.txt`		05 9. サーバーに置き、Windows サービスにする
表を新しくする	`python manage.py migrate`		＋ 合わせて覚える
静的ファイルを集める	`python manage.py collectstatic --noinput`		05 6. アプリの作り
本番の設定の確かめ	`python manage.py check --deploy`		05 8. HTTP で動かすための設定
Django を本番用のサーバーで動かす	`python -m waitress --listen=127.0.0.1:8081 config.wsgi:application`		05 6. アプリの作り
動いているか確かめる	`Invoke-WebRequest http://127.0.0.1:8080/health -UseBasicParsing`		00 5. Web アプリの仕組み
秘密の値を環境変数で読む（Python）	`os.environ.get(name, default)`		05 6. アプリの作り
設定の値が無ければ止める	`raise RuntimeError(f"環境変数 {name} がありません")`		05 6. アプリの作り

やりたいこと	書き方	ひとこと	出てくる所
受け付ける名前（Django）	`ALLOWED_HOSTS = env("DJANGO_ALLOWED_HOSTS").split(",")`		05 6. アプリの作り
CSRF の札を付けて送る（JavaScript）	`headers: { "X-CSRFToken": csrfToken() }`		05 6. アプリの作り

正規表現と日時の書式

文字の形を表す書き方（正規表現）と、日時を文字にする書き方です。

やりたいこと	書き方	ひとこと	出てくる所
形に合うか（PowerShell）	`$line -match '^\s*host'`		01 3. PowerShell の基本
行の頭・行の終わり	`^` `$`		04 5. データベース

数字・空白・英数字	`\d`
	`\s`
	`\w`
			06 3. 記録

くり返し	`*`
	`+`
	`{6}`
			04 5. データベース

どちらか	`api/|admin/`		05 6. アプリの作り
まとまり	`(.*)`	{R:1} や $1 で使う	05 6. アプリの作り
～でない時だけ	`^(?!api/|admin/).*$`		05 6. アプリの作り

年・月・日・時・分	`yyyy`
	`MM`
	`dd`
	`HH`
	`mm`
		MM は月、mm は分	04 11. 毎晩のバックアップ

やりたいこと	書き方	ひとこと	出てくる所
置きかえる	`$s -replace '\s+', ' '`		＋ 合わせて覚える